1) Цель

   Знакомство с многопоточным программированием и методами синхронизации потоков средствами POSIX.

2) Задание

   a) С помощью таблицы вариантов заданий выбрать граф запуска потоков в соответствии с номером варианта. Вершины графа являются точками запуска/завершения потоков, дугами обозначены сами потоки. Длину дуги следует интерпретировать как ориентировочное время выполнения потока. В процессе своей работы каждый поток должен в цикле выполнять два действия: выводить букву имени потока в консоль; вызывать функцию computation() для выполнения вычислений, требующих задействования ЦП на длительное время. Эта функция уже написана и подключается из заголовочного файла lab2.h, изменять ее не следует.

   b) В соответствии с вариантом выделить на графе две группы с выполняющимися параллельно потоками. В первой группе потоки не синхронизированы, параллельное выполнение входящих в группу потоков происходит за счет планировщика задач (см. примеры 1 и 2). Вторая группа синхронизирована семафорами и потоки внутри группы выполняются в строго зафиксированном порядке: входящий в групу поток передает управление другому потоку после каждой итерации цикла (см. пример 3 и задачу производителя и потребителя). Таким образом потоки во второй группе выполняются в строгой очередности.

   c) С использованием средств POSIX реализовать программу для последовательно-параллельного выполнения потоков в ОС Linux или Mac OS X. Запрещается использовать какие-либо библиотеки и модули, решающие задачу кроссплатформенной разработки многопоточных приложений (std::thread, Qt Thread, Boost Thread и т.п.), а также функции приостановки выполнения программы за исключением pthread_yield(). Для этого необходимо написать код в файле lab2.cpp:

      (i) Функция unsigned int lab2_thread_graph_id() должна возвращать номер графа запуска потоков, полученный из таблицы вариантов заданий.

      (ii) Функция const char* lab2_unsynchronized_threads() должна возвращать строку, состоящую из букв потоков, выполняющихся параллельно без синхронизации (см. примеры в файлах lab2.cpp и lab2_ex.cpp).

      (iii) Функция const char* lab2_sequential_threads() должна возвращать строку, состоящую из букв потоков, выполняющихся параллельно в строгой очередности друг за другом (см. примеры в файлах lab2.cpp и lab2_ex.cpp).

(iv) Функция int lab2_init() заменяет собой функцию main(). В ней необходимо реализовать запуск потоков, инициализацию вспомогательных переменных (мьютексов, семафоров и т.п.). Перед выходом из функции lab2_init() необходимо убедиться, что все запущенные потоки завершились. Возвращаемое значение: 0 - работа функции завершилась успешно, любое другое числовое значение - при выполнении функции произошла критическая ошибка.

(v) Добавить любые другие необходимые для работы программы функции, переменные и подключаемые файлы.

(vi) Создавать функцию main() не нужно. В проекте уже имеется готовая функция main(), изменять ее нельзя. Она выполняет единственное действие: вызывает функцию lab2_init().

Интервалы с несинхронизированными потоками - bcde

Интервалы с чередованием потоков - deg

Не следует изменять какие-либо файлы, кроме lab2.cpp. Также не следует создавать новые файлы и писать в них код, поскольку код из этих файлов не будет использоваться во время тестирования.

d) Подготовить отчет о выполнении лабораторной работы и загрузить его под именем report.pdf в репозиторий. В случае использования системы компьютерной верстки LaTeX также загрузить исходный файл report.tex.

3) Граф запуска потоков

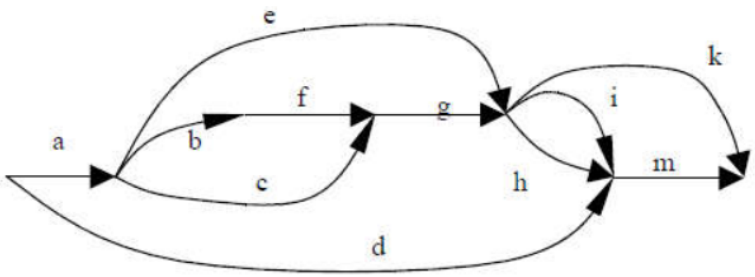


Рис. 1 – Граф запуска потоков

4) Результат выполнения работы

По результатам лабораторной работы была получена программа запуска потоков с синхронизацией средствами POSIX, которая прошла тестирование на работоспособность посредством автоматизированных тестов.

5) Исходный код

lab2.cpp

```
#include "lab2.h"
#include <cstring>
#include <semaphore.h>

#define NUMBER_OF_THREADS 11

// Потоки
pthread_t tid[NUMBER_OF_THREADS];
// Мьютекс критической секции
pthread_mutex_t lock;
// Семафоры
sem_t semC,semD,semE,semG,semK;

int err;

unsigned int lab2_thread_graph_id()
{
    return 11;
}

const char* lab2_unsynchronized_threads()
{
    return "bcde";
}

const char* lab2_sequential_threads()
{
    return "deg";
}

// Функции потоков
void *thread_a(void *ptr);
void *thread_b(void *ptr);
void *thread_c(void *ptr);
void *thread_d(void *ptr);
void *thread_e(void *ptr);
void *thread_f(void *ptr);
void *thread_g(void *ptr);
void *thread_h(void *ptr);
void *thread_i(void *ptr);
void *thread_k(void *ptr);
```

```
void *thread_m(void *ptr);

void *thread_a(void *ptr)
{
    for (int i = 0; i < 3; ++i) {
        pthread_mutex_lock(&lock);
        std::cout << "a" << std::flush;
        pthread_mutex_unlock(&lock);
        computation();
    }

    sem_post(&semD);

    return  ptr;
}

void *thread_b(void *ptr)
{
    for (int i = 0; i < 3; ++i) {
        pthread_mutex_lock(&lock);
        std::cout << "b" << std::flush;
        pthread_mutex_unlock(&lock);
        computation();
    }

    sem_post(&semD);

    return  ptr;
}

void *thread_c(void *ptr)
{
    for (int i = 0; i < 3; ++i) {
        pthread_mutex_lock(&lock);
        std::cout << "c" << std::flush;
        pthread_mutex_unlock(&lock);
        computation();
    }

    sem_wait(&semC);
    err = pthread_create(&tid[5], NULL, thread_f, NULL);
```

```
    if (err != 0)
    {
        std::cerr << "Can't create thread. Error: " << strerror(err) << std::endl;
    }

    for (int i = 0; i < 3; ++i) {
        pthread_mutex_lock(&lock);
        std::cout << "c" << std::flush;
        pthread_mutex_unlock(&lock);
        computation();
    }

    return  ptr;
}

void *thread_d(void *ptr)
{
    for (int i = 0; i < 3; ++i) {
        pthread_mutex_lock(&lock);
        std::cout << "d" << std::flush;
        pthread_mutex_unlock(&lock);
        computation();
    }

    sem_wait(&semD);

    err = pthread_create(&tid[4], NULL, thread_e, NULL);
    if (err != 0)
    {
        std::cerr << "Can't create thread. Error: " << strerror(err) << std::endl;
    }
    err = pthread_create(&tid[1], NULL, thread_b, NULL);
    if (err != 0)
    {
        std::cerr << "Can't create thread. Error: " << strerror(err) << std::endl;
    }
    err = pthread_create(&tid[2], NULL, thread_c, NULL);
    if (err != 0)
    {
        std::cerr << "Can't create thread. Error: " << strerror(err) << std::endl;
    }
```

```
for (int i = 0; i < 3; ++i) {
    pthread_mutex_lock(&lock);
    std::cout << "d" << std::flush;
    pthread_mutex_unlock(&lock);
    computation();
}

sem_wait(&semD);
sem_post(&semE);

for (int i = 0; i < 3; ++i) {
    pthread_mutex_lock(&lock);
    std::cout << "d" << std::flush;
    pthread_mutex_unlock(&lock);
    computation();
}

for (int i = 0; i < 3; ++i) {
    sem_wait(&semD);
    pthread_mutex_lock(&lock);
    std::cout << "d" << std::flush;
    pthread_mutex_unlock(&lock);
    computation();
    sem_post(&semE);
}

sem_wait(&semD);

for (int i = 0; i < 3; ++i) {
    pthread_mutex_lock(&lock);
    std::cout << "d" << std::flush;
    pthread_mutex_unlock(&lock);
    computation();
}

pthread_join(tid[7], NULL);
pthread_join(tid[8], NULL);
sem_post(&semK);

err = pthread_create(&tid[10], NULL, thread_m, NULL);
```

```
    if (err != 0)
    {
        std::cerr << "Can't create thread. Error: " << strerror(err) << std::endl;
    }

    pthread_join(tid[10], NULL);
    pthread_join(tid[9], NULL);

    return  ptr;
}

void *thread_e(void *ptr)
{
    for (int i = 0; i < 3; ++i) {
        pthread_mutex_lock(&lock);
        std::cout << "e" << std::flush;
        pthread_mutex_unlock(&lock);
        computation();
    }

    sem_wait(&semE);
    sem_post(&semC);

    for (int i = 0; i < 3; ++i) {
        pthread_mutex_lock(&lock);
        std::cout << "e" << std::flush;
        pthread_mutex_unlock(&lock);
        computation();
    }

    pthread_join(tid[2], NULL);
    pthread_join(tid[5], NULL);
    sem_post(&semD);
    err = pthread_create(&tid[6], NULL, thread_g, NULL);
    if (err != 0)
    {
        std::cerr << "Can't create thread. Error: " << strerror(err) << std::endl;
    }

    for (int i = 0; i < 3; ++i) {
        sem_wait(&semE);
```

```
        pthread_mutex_lock(&lock);
        std::cout << "e" << std::flush;
        pthread_mutex_unlock(&lock);
        computation();
        sem_post(&semG);
    }

    return  ptr;
}

void *thread_f(void *ptr)
{
    for (int i = 0; i < 3; ++i) {
        pthread_mutex_lock(&lock);
        std::cout << "f" << std::flush;
        pthread_mutex_unlock(&lock);
        computation();
    }

    return  ptr;
}

void *thread_g(void *ptr)
{
    for (int i = 0; i < 3; ++i) {
        sem_wait(&semG);
        pthread_mutex_lock(&lock);
        std::cout << "g" << std::flush;
        pthread_mutex_unlock(&lock);
        computation();
        sem_post(&semD);
    }

    err = pthread_create(&tid[7], NULL, thread_h, NULL);
    if (err != 0)
    {
        std::cerr << "Can't create thread. Error: " << strerror(err) << std::endl;
    }

    err = pthread_create(&tid[8], NULL, thread_i, NULL);
    if (err != 0)
```

```
    {
        std::cerr << "Can't create thread. Error: " << strerror(err) << std::endl;
    }

    err = pthread_create(&tid[9], NULL, thread_k, NULL);
    if (err != 0)
    {
        std::cerr << "Can't create thread. Error: " << strerror(err) << std::endl;
    }
    return  ptr;
}

void *thread_h(void *ptr)
{
    for (int i = 0; i < 3; ++i) {
        pthread_mutex_lock(&lock);
        std::cout << "h" << std::flush;
        pthread_mutex_unlock(&lock);
        computation();
    }

    return  ptr;
}

void *thread_i(void *ptr)
{
    for (int i = 0; i < 3; ++i) {
        pthread_mutex_lock(&lock);
        std::cout << "i" << std::flush;
        pthread_mutex_unlock(&lock);
        computation();
    }

    return  ptr;
}

void *thread_k(void *ptr)
{
    for (int i = 0; i < 3; ++i) {
        pthread_mutex_lock(&lock);
        std::cout << "k" << std::flush;
```

```
        pthread_mutex_unlock(&lock);
        computation();
    }

    sem_wait(&semK);

    for (int i = 0; i < 3; ++i) {
        pthread_mutex_lock(&lock);
        std::cout << "k" << std::flush;
        pthread_mutex_unlock(&lock);
        computation();
    }

    return  ptr;
}

void *thread_m(void *ptr)
{
    for (int i = 0; i < 3; ++i) {
        pthread_mutex_lock(&lock);
        std::cout << "m" << std::flush;
        pthread_mutex_unlock(&lock);
        computation();
    }

    return  ptr;
}

int lab2_init()
{
    // Создание мьютекса
    if (pthread_mutex_init(&lock, NULL) != 0) {
        std::cerr << "Mutex init failed" << std::endl;
        return 1;
    }

    // Создание семафоров
    if ( sem_init(&semC, 0, 0) != 0 ) {
        std::cerr << "Semaphore C init failed" << std::endl;
        return 1;
    }
```

```
if ( sem_init(&semD, 0, 0) != 0 ) {
    std::cerr << "Semaphore D init failed" << std::endl;
    return 1;
}

if ( sem_init(&semE, 0, 0) != 0 ) {
    std::cerr << "Semaphore E init failed" << std::endl;
    return 1;
}

if ( sem_init(&semG, 0, 0) != 0 ) {
    std::cerr << "Semaphore G init failed" << std::endl;
    return 1;
}

if ( sem_init(&semK, 0, 0) != 0 ) {
    std::cerr << "Semaphore K init failed" << std::endl;
    return 1;
}

// Создание исходных потоков
err = pthread_create(&tid[0], NULL, thread_a, NULL);
if (err != 0)
{
    std::cerr << "Can't create thread. Error: " << strerror(err) << std::endl;
}
err = pthread_create(&tid[3], NULL, thread_d, NULL);
if (err != 0)
{
    std::cerr << "Can't create thread. Error: " << strerror(err) << std::endl;
}

// Ожидание окончания A и D
pthread_join(tid[0], NULL);
pthread_join(tid[3], NULL);

pthread_mutex_destroy(&lock);
sem_destroy(&semD);

return 0;
```

```
}
```

6) Выводы

В процессе выполнения лабораторной работы были получены практические навыки работы с потоками и их синхронизацией средствами POSIX в операционной системе Ubuntu.